

Week14예습과제_박지원

1. 추천 시스템의 배경과 역할

추천 시스템은 사용자의 과거 행동(구매, 평점, 클릭, 시청 등) 데이터를 분석해 사용자가 좋아할 만한 아이템을 예측해 제시하는 시스템이다. 아마존, 유튜브, 넷플릭스 등 대형 플랫폼들이 추천 엔진 고도화에 막대한 투자를 하는 이유는, 좋은 추천이 사용자 체류 시간과 매출을 직접적으로 끌어올리기 때문이다.

온라인 스토어는 상품 수가 매우 많아 사용자가 선택의 압박을 느끼기 쉬운데, 추천 시스템은 이 부담을 줄여주는 핵심 장치 역할을 한다. 추천에 활용되는 데이터로는 구매 이력, 장바구니/탐색 이력, 평점, 사용자가 직접 입력한 선호 정보, 클릭 로그 등이 있다.

2. 추천 시스템의 분류

추천 시스템은 크게 두 갈래로 나뉜다.

(1) 콘텐츠 기반 필터링(Content-based Filtering)

아이템 자체의 속성(장르, 감독, 배우, 키워드, 설명 텍스트 등)을 비교해, 사용자가 좋아한 아이템과 콘텐츠적으로 유사한 다른 아이템을 추천하는 방식이다. 즉 "이 영화를 좋아했다면 비슷한 장르/감독/배우의 다른 영화도 좋아할 것"이라는 직관에 기반한다.

(2) 협업 필터링(Collaborative Filtering)

아이템의 속성과는 무관하게, 사용자들의 평가·행동 패턴만을 가지고 추천하는 방식이다. 협업 필터링은 다시 두 가지로 나뉜다.

최근접 이웃(Nearest Neighbor, 메모리 기반) 협업 필터링

사용자 기반: 나와 평가 패턴이 비슷한 다른 사용자를 찾아, 그 사용자가 좋아한 아이템을 추천("당신과 비슷한 고객들이 구매한 상품")

아이템 기반: 특정 아이템에 대한 여러 사용자의 평가 패턴이 비슷한 다른 아이템을 추천("이 상품을 선택한 사람들이 구매한 다른 상품"). 일반적으로 아이템 기반이 사용자 기반보다 정확도가 더 높다고 알려져 있는데, 이는 유명 아이템은 취향과 무관하게 누구나 평가하는 경우가 많아 사용자 간 유사도 비교가 왜곡되기 쉽기 때문이다.

잠재 요인(Latent Factor) 협업 필터링

사용자-아이템 평점 행렬 속에 숨겨진 "잠재 요인"(예: 영화라면 장르별 선호도 같은 잠재적 특성)을 추출해, 이를 기반으로 미평가 항목의 평점을 예측하는 방식이다. 넷플릭스 추천 경연대회에서 행렬 분해 기법이 우승하면서 널리 알려졌고, 현재 대부분의 온라인 서비스가 이 방식을 채택하고 있다.

콘텐츠 기반과 협업 기반은 상호 배타적이지 않으며, 최근에는 둘을 결합한 하이브리드 방식도 많이 쓰인다.

3. 사용자-아이템 평점 행렬

협업 필터링의 핵심 데이터 구조는 행을 사용자, 열을 아이템으로 하고 각 셀에 평점이 들어가는 행렬이다. 실제 데이터는 보통 (사용자ID, 아이템ID, 평점)의 로우 레벨 형태로 저장되어 있으므로, `pivot_table()` 같은 함수로 행렬 형태로 변환하는 전처리가 필요하다. 대부분의 사용자는 전체 아이템 중 일부에만 평점을 남기기 때문에 이 행렬은 결측값이 매우 많은 희소 행렬(Sparse Matrix) 특성을 가진다.

추천 시스템에서 유사도를 측정할 때는 텍스트 분석에서와 마찬가지로 코사인 유사도가 주로 사용된다. 두 벡터(사용자 벡터 또는 아이템 벡터) 사이의 방향이 얼마나 비슷한지를 측정하는 방식으로, 희소하고 차원이 큰 데이터에 적합하기 때문이다.

4. 잠재 요인 협업 필터링과 행렬 분해(Matrix Factorization)

잠재 요인 협업 필터링의 핵심 아이디어는, $M \times N$ 크기의 사용자-아이템 평점 행렬 R 을 더 작은 차원 K 를 갖는 두 행렬의 곱으로 근사하는 것이다.

$$R(M \times N) \approx P(M \times K) \times Q.T(K \times N)$$

P: 사용자-잠재요인 행렬 (각 사용자가 K 개의 잠재 요인에 대해 갖는 선호도)

Q: 아이템-잠재요인 행렬 (각 아이템이 K 개의 잠재 요인에 대해 갖는 특성값), Q.T는 그 전치행렬

여기서 "잠재 요인"이 구체적으로 무엇을 의미하는지는 알고리즘이 명시적으로 정의해주지 않는다. 다만 영화 평점 데이터의 경우 직관적으로 장르별 선호도/특성 같은 것으로 해석할 수 있다. P의 한 사용자 벡터와 Q.T의 한 아이템 벡터를 내적하면 해당 사용자가 해당 아이템에 줄 것으로 예측되는 평점이 나온다. 이 방식의 장점은, 평점이 없는(결측) 칸에 대해서도 동일한 내적 연산으로 예측값을 채울 수 있다는 것이다.

문제는 R 행렬에 결측값이 많아 일반적인 SVD(특이값 분해)를 직접 적용할 수 없다는 점이다. 그래서 실무에서는 **확률적 경사 하강법(SGD)**이나 ALS(Alternating Least Squares) 방식으로 P, Q를 근사적으로 찾는다.

5. SGD 기반 행렬 분해의 절차

P, Q를 임의의 값(정규분포 난수 등)으로 초기화한다.

현재 P, Q로 예측 행렬을 계산하고, 실제 평점과의 오차(e)를 구한다.

오차를 줄이는 방향으로 L2 정규화를 반영해 P, Q의 해당 행을 업데이트한다.

정해진 반복 횟수(steps) 동안 2~3을 반복하며 점진적으로 오차를 줄여간다.

업데이트 시 정규화(Regularization) 항을 추가하는 이유는 학습 과정에서 과적합을 방지하기 위함이다. 학습률(learning rate)과 정규화 계수(λ)는 하이퍼파라미터로서 수렴 속도와 안정성에 영향을 준다.

실습에서는 결측값을 포함한 작은 행렬 R 을 만들고, P·Q를 무작위 초기화한 뒤 1000회 반복 학습시켜 RMSE가 점차 줄어드는 것을 확인했다. 학습이 끝난 P, Q의 내적으로 만든 예측 행렬은 원래 값이 있던 칸은 거의 그대로 유지하면서, 결측이었던 칸에는 새로운 예측값이 채워진다.

6. 콘텐츠 기반 필터링 실습 (TMDB 5000)

장르 정보를 활용해 콘텐츠 기반 필터링을 구현하는 과정은 다음 흐름을 따른다.

데이터 전처리: genres, keywords 칼럼은 딕셔너리 리스트가 문자열로 저장된 형태이므로, ast.literal_eval()로 실제 객체로 변환한 뒤 'name' 키 값만 추출해 리스트로 만든다.

피처 벡터화: 장르 리스트를 공백으로 구분된 문자열로 합친 뒤, CountVectorizer로 단어(장르명) 단위 피처 벡터를 생성한다.

유사도 계산: 생성된 피처 벡터 행렬에 cosine_similarity를 적용해 영화 간 장르 유사도 행렬을 만든다.

추천: 특정 영화와 장르 유사도가 높은 영화들을 추출해 추천한다.

다만 단순히 장르 유사도만으로 추천하면, 평가 수가 극히 적어 평점이 왜곡된 영화(예: 평가 1~2명인데 만점)가 섞여 들어오는 문제가 생긴다. 이를 보정하기 위해 IMDB 방식의 가중 평점(Weighted Rating) 공식을 적용한다.

가중 평점 = $(v/(v+m)) \times R + (m/(v+m)) \times C$

v: 해당 영화의 투표 수, R: 해당 영화의 평균 평점

m: 평점을 신뢰할 만하다고 보는 최소 투표 수 기준(전체 투표수의 상위 60% 등으로 설정)

C: 전체 영화의 평균 평점

이렇게 하면 투표 수가 적은 영화는 전체 평균(C) 쪽으로 평점이 보정되고, 투표 수가 많은 영화는 자신의 실제 평점(R)에 가깝게 유지된다. 최종적으로는 장르 유사도가 높은 영화를 넉넉히(예: 추천 개수의 2배) 추려낸 뒤, 그 후보군 안에서 가중 평점이 높은 순으로 다시 정렬해 최종 추천 목록을 만드는 2단계 방식이 더 합리적인 결과를 낸다.

7. 아이템 기반 최근접 이웃 협업 필터링 실습 (MovieLens)

이 실습은 장르 같은 콘텐츠 정보를 전혀 쓰지 않고, 오직 사용자들의 평점 패턴만으로 추천하는 방식이다.

ratings 데이터(사용자ID-영화ID-평점)와 movies 데이터(영화ID-제목)를 조인해 영화 제목을 얻고, pivot_table()로 "사용자 × 영화" 평점 행렬을 만든다. 결측은 0으로 채운다.

영화 간 유사도를 구하려면 행렬의 행이 영화가 되어야 하므로, 행렬을 **전치

(transpose)**한 뒤 cosine_similarity를 적용한다. (전치하지 않으면 영화 유사도가 아니라 사용자 유사도가 나오므로 주의해야 한다.)

이렇게 얻은 아이템(영화) 간 유사도 행렬을 이용해, 특정 영화와 가장 비슷한 영화들을 단순 추출할 수 있다.

더 나아가, 개인화된 예측 평점을 계산하는 공식을 적용한다.

예측평점($\hat{R}_{ui}$) = $\sum(\text{유사도} \times \text{실제평점}) / \sum|\text{유사도}|$

즉 사용자가 실제로 평가한 다른 영화들의 점수를, 그 영화들과 추천 대상 영화 사이의 유사도로 가중평균을 낸 값이다.

처음에는 모든 영화와의 유사도를 사용했으나, 이는 관련성이 낮은 영화까지 다 반영되어 예측 오차(MSE)가 크게 나왔다. 그래서 Top-N(가장 유사도가 높은 N개) 영화만 사용하도록 로직을 바꾸자 MSE가 크게 개선되었다(약 9.89 → 약 3.69). 이는 너무 많은 정보를 평균 내면 오히려 신호가 희석된다는 점을 보여주는 결과다.

마지막으로, 특정 사용자가 아직 평가하지 않은 영화 목록을 추출하고, 그 목록 중 예측 평점이 가장 높은 영화들을 추천하는 개인화 추천 함수를 구현한다.

1. 두 실습의 비교와 시사점

콘텐츠 기반 필터링은 새로운 아이템(아직 아무도 평가하지 않은 아이템)에도 추천이 가능하다는 장점이 있지만, 장르처럼 한정된 속성만으로는 영화가 주는 분위기, 연출, 디테일한 매력을 다 반영하지 못하는 한계가 있다.

아이템 기반 협업 필터링은 실제 사용자들의 평가 패턴을 반영하므로 더 정교한 추천이 가능하지만, 신규 아이템처럼 평가 데이터가 없는 경우에는 추천이 어렵다는 "콜드 스타트(cold start)" 문제가 있다.

두 접근을 결합하는 하이브리드 방식이 실무에서 점점 더 널리 쓰이는 이유가 여기에 있다.

1. 평가 지표 관련 주의점

예측 평점의 정확도는 MSE(혹은 RMSE)로 측정하는데, 이때 주의할 점은 사용자가 실제로 평가한 항목에 대해서만 오차를 계산해야 한다는 것이다. 결측값을 0으로 채운 뒤 그 0까지 포함해서 오차를 계산하면, 단순히 "평가하지 않은 항목"과 "낮은 평점을 준 항목"이 혼동되어 평가 지표가 왜곡된다. 따라서 `nonzero()` 등으로 실제 평점이 존재하는 위치만 골라내 비교하는 과정이 반드시 필요하다.